

Exemples d'utilisation en création d'interfaces

```
class CarouselImage {
  #conteneurHTML;
  #listeImages;
  #index;
  #element;

  constructor(conteneurHTML, listeImages) {
    this.#conteneurHTML = conteneurHTML;
    this.#listeImages = listeImages;
    this.#index = 0;
    this.#element = null;
    this.injecterHTML();
  }

  injecterHTML() {
    const gabarit = `
      <div class="carousel" data-carousel>
        <button class="prev" data-direction="-1">Précédent</button>
        <img class="image-container" src="" alt="Image carousel">
        <button class="next" data-direction="1">Suivant</button>
      </div>
    `;

    this.#conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
    this.#element = this.#conteneurHTML.lastElementChild;

    this.#element.addEventListener("click", this.#onClicBouton.bind(this));
    this.#afficherImage(this.#index);
  }

  #onClicBouton(evenement) {
    const declencheur = evenement.target;

    if (declencheur.closest("[data-direction='-1']")) {
      this.#reculer();
    } else if (declencheur.closest("[data-direction='1']")) {
      this.#avancer();
    }
  }

  #afficherImage(index) {
    const imageContainer = this.#element.querySelector(".image-container");
    imageContainer.src = this.#listeImages[index];
    imageContainer.alt = `Image ${index + 1}`;
  }

  #avancer() {
    this.#index = (this.#index + 1) % this.#listeImages.length;
    this.#afficherImage(this.#index);
  }
}
```

```

    }

    #reculer() {
        this.#index = (this.#index - 1 + this.#listeImages.length) %
this.#listeImages.length;
        this.#afficherImage(this.#index);
    }
}

```

Toast

```

class Toast {
    #conteneurHTML;
    #element;
    #message;

    constructor(conteneurHTML, message) {
        this.#conteneurHTML = conteneurHTML;
        this.#element = null;
        this.#message = message;
    }

    afficher(message) {
        const gabarit = `
            <div class="toast">
                <p>${message}</p>
            </div>
        `;

        this.#conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
        this.#element = this.#conteneurHTML.lastElementChild;
        this.#element.addEventListener("click", this.#fermer.bind(this));

        setTimeout(this.#fermer.bind(this), 3000);
    }

    #fermer() {
        if (this.#element) {
            this.#element.remove();
            this.#element = null;
        }
    }
}

```

Pagination

```

class Pagination {
    #conteneurHTML;
    #element;
    #pageCourante;

```

```

#totalPages;

constructor(conteneurHTML, totalPages, pageCourante = 1) {
  this.#conteneurHTML = conteneurHTML;
  this.#element = null;
  this.#pageCourante = Math.min(Math.max(pageCourante, 1), totalPages); //
Assure que la page courante est entre 1 et totalPages
  this.#totalPages = totalPages;
  this.#injecterHTML();
  this.#majAffichage();
  // console.log(`Pagination créée :
  ${this.#pageCourante}/${this.#totalPages}`);
}

get pageCourante() {
  return this.#pageCourante;
}

set pageCourante(nouvellePage) {
  this.#pageCourante = Math.min(Math.max(nouvellePage, 1),
this.#totalPages);
  this.#majAffichage();
}

#injecterHTML() {
  const gabarit = `
    <div class="pagination">
      <button class="prev" data-direction="-1">Précédent</button>
      <span class="page-info">Page ${this.#pageCourante} sur
  ${this.#totalPages}</span>
      <button class="next" data-direction="1">Suivant</button>
    </div>
  `;

  this.#conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
  this.#element = this.#conteneurHTML.lastElementChild;

  this.#element.addEventListener("click", this.#onClicBouton.bind(this));
}

#onClicBouton(evenement) {
  const declencheur = evenement.target;

  if (declencheur.closest("[data-direction='-1']")) {
    this.#pageCourante = Math.max(1, this.#pageCourante - 1);
  } else if (declencheur.closest("[data-direction='1']")) {
    this.#pageCourante = Math.min(this.#totalPages, this.#pageCourante +
1);
  }

  this.#majAffichage();
}

#majAffichage() {
  const pageInfo = this.#element.querySelector(".page-info");

```

```

        pageInfo.textContent = `Page ${this.#pageCourante} sur
        ${this.#totalPages}`;

        this.#element.querySelector("[data-direction='-1']").disabled =
        this.#pageCourante === 1;
        this.#element.querySelector("[data-direction='1']").disabled =
        this.#pageCourante === this.#totalPages;
    }
}

```

Widget d'un tableau de bord et d'un compteur qui hérite de Widget

```

class Widget {
    #conteneurHTML;
    #elementHTML;
    #nom;
    #largeur;
    #hauteur;
    #id;
    #estFocus;

    constructor(conteneurHTML, nom, largeur = 2, hauteur = 2) {
        this.#conteneurHTML = conteneurHTML;
        this.#nom = nom;
        this.#largeur = largeur;
        this.#hauteur = hauteur;
        this.#id = Math.random().toString(36).substr(2, 9);
        this.#elementHTML = null;
        this.#estFocus = false;
        this.#injecterHTML();
    }

    get _elementHTML() {
        return this.#elementHTML;
    }

    #injecterHTML() {
        const gabarit = `
            <div data-widget class="widget" id="widget-${this.#id}"
            style="grid-column: span ${this.#largeur}; grid-row: span
            ${this.#hauteur};">
                <h3>Widget ${this.#nom}</h3>
                <p data-contenu-widget></p>
            </div> `;
        this.#conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
        this.#elementHTML = this.#conteneurHTML.lastElementChild;

        this.#elementHTML.addEventListener("click", this.#onClic.bind(this));
    }

    #onClic() {
        this.#estFocus = !this.#estFocus;
    }
}

```

```

        this.#elementHTML.classList.toggle("highlight", this.#estFocus);
        this.#focus();
    }

    #focus() {
        this.#elementHTML.scrollIntoView({ behavior: "smooth", block: "center" });
    }
}

class Compteur extends Widget {
    #temps = 0;
    #intervalle = null;

    #contenuHTMLWidget = null;

    constructor(conteneurHTML, nom, largeur = 2, hauteur = 2) {
        super(conteneurHTML, nom, largeur, hauteur);
        this.#temps = 0;
        this.#intervalle = null;
        this.#contenuHTMLWidget = null;

        this.#injecterHTML();
    }

    /**
     * Retourne le temps formaté en mm:ss
     */
    get temps() {
        const minutes = Math.floor(this.#temps / 60);
        const secondes = this.#temps % 60;

        const tempsFormatte = `${minutes.toString().padStart(2, "0")}:${secondes.toString().padStart(2, "0")}`;

        return tempsFormatte;
    }

    /**
     * Génère le HTML du compteur et attache les événements
     */
    #injecterHTML() {
        const gabarit = `<div data-compteur>Temps écoulé : <span data-conteneur-temps>${this.temps}</span></div>`;

        this._elementHTML.insertAdjacentHTML("beforeend", gabarit);
        this.#contenuHTMLWidget = this._elementHTML.lastElementChild;

        this._elementHTML.addEventListener("click", this.demarrer.bind(this));
    }

    /**
     * Met à jour le HTML du compteur
     */
    #mettreAJourHTML() {
        this.#contenuHTMLWidget.querySelector("[data-conteneur-

```

```

temps]").textContent = this.temps;
}

/**
 * Démarre le compteur
 */
demarrer() {
    this.arreter();

    this.#intervalle = setInterval(() => {
        this.#temps++;
        this.#mettreAJourHTML();
    }, 1000);
}

/**
 * Arrête le compteur
 */
arreter() {
    clearInterval(this.#intervalle);
    this.#intervalle = null;
    this.#temps = 0;
}
}

```

Tableau de bord qui gère plusieurs widgets

```

class App {
    static #instance = null;
    #conteneurHTML;
    #widgets;

    constructor(conteneurHTML) {
        //Ne permet qu'une seule instance de App. C'est le patron de conception
        Singleton.
        if (App.#instance !== null) {
            console.warn("Une instance de App existe déjà.");
            return App.#instance;
        } else {
            App.#instance = this;
        }

        this.#conteneurHTML = conteneurHTML;
        this.#widgets = [];
    }

    ajouterWidget(widget) {
        this.#widgets.push(widget);
    }

    demarrer() {
        this.#widgets.forEach((widget) => {

```

```

        if (widget instanceof Compteur) {
            widget.demarrer();
        }
    });
}

arreter() {
    this.#widgets.forEach((widget) => {
        if (widget instanceof Compteur) {
            widget.arreter();
        }
    });
}
}

// Dans le fichier principal, on crée une instance de App et on ajoute des widgets
de cette façon
const app = new App(document.querySelector("#widget-container"));

app.ajouterWidget(new Compteur(document.querySelector("#widget-container"),
"Compteur 1", 2, 2));
app.ajouterWidget(new Compteur(document.querySelector("#widget-container"),
"Compteur 2", 4, 4));

//Déclenche tous les compteurs en même temps
app.demarrer();

setTimeout(() => {
    //Arrête tous les compteurs en même temps après 5 secondes
    app.arreter();
}, 5000);

```

Filtres

```

class Filtre {
    #conteneurHTML;
    #elementHTML;
    #listeProduits;
    #listeFiltree;
    #tri;
    #filtre;
    #direction;

    constructor(conteneurHTML, listeProduits = []) {
        this.#conteneurHTML = conteneurHTML;
        this.#elementHTML = null;

        this.#filtre = "tout";
        this.#tri = "alphabetique";
        this.#direction = "asc";

        this.#listeProduits = listeProduits;
    }
}

```

```

        this.#listeFiltree = listeProduits;

        this.#injecterHTML();
        this.#afficherProduits();
    }

    get listeFiltree() {
        return this.#listeFiltree;
    }

    #injecterHTML() {
        const gabarit = `
            <div data-filtres>
                <div class="bouton" data-filtre="tout">Tout afficher</div>
                <div class="bouton" data-filtre="date">Afficher uniquement les
nouveauautés</div>
                <div class="bouton" data-tri="alphabetique" data-
direction="asc">Nom A-Z</div>
                <div class="bouton" data-tri="alphabetique" data-
direction="desc">Nom Z-A</div>
            </div>
        `;

        this.#conteneurHTML.querySelector("[data-conteneur-
filtres]").insertAdjacentHTML("beforeend", gabarit);
        this.#elementHTML = this.#conteneurHTML.querySelector("[data-conteneur-
filtres]").lastElementChild;
        this.#elementHTML.addEventListener("click",
this.#appliquerFiltre.bind(this));
    }

    #afficherProduits() {
        this.#conteneurHTML.querySelector("[data-conteneur-produits]").innerHTML =
"";

        const gabarit = this.#listeFiltree
            .map(
                (produit) => `
                    <div class="produit">
                        <h2>${produit.nom}</h2>
                        <p>Prix : ${produit.prix} €</p>
                        <p>Date d'ajout : ${new
Date(produit.date).toLocaleDateString()}</p>
                    </div>
                `
            )
            .join("");

        this.#conteneurHTML.querySelector("[data-conteneur-
produits]").insertAdjacentHTML("beforeend", gabarit);
    }

    #appliquerFiltre(evenement) {
        const declencheur = evenement.target;

```

```

    if (declencheur.closest("[data-filtre]")) {
        this.#filtre = declencheur.dataset.filtre || "tout";
        this.#filtrer();
    }

    if (declencheur.closest("[data-tri]")) {
        this.#tri = declencheur.dataset.tri || "alphabetique";
        this.#direction = declencheur.dataset.direction || "asc";
        this.#trier();
    }
    this.#afficherProduits();
}

#filtrer() {
    if (!this.#listeProduits) return;

    this.#listeFiltree = this.#listeProduits.filter((produit) => {
        if (this.#filtre === "tout") return true;

        // Retourne les produits de moins de 7 jours
        if (this.#filtre === "date")
            return new Date(produit.date) >= new Date(Date.now() - 7 * 24 * 60
* 60 * 1000);
    });
}

#trier() {
    if (!this.#listeProduits) return;

    this.#listeFiltree.sort((a, b) => {
        if (this.#tri === "alphabetique") {
            return this.#direction === "asc" ? a.nom.localeCompare(b.nom) :
b.nom.localeCompare(a.nom);
        }
    });
}
}

```